

Práctica 1

Búsqueda no informada, informada y con adversarios

Laboratorio de Inteligencia Artificial – Curso 2026-2027

Fecha de publicación: 14 de septiembre

Fecha tope para participar en cada torneo: 7, 14 y 21 de octubre, a las 20:00.

Entrega: martes 20 o jueves 22 de octubre antes de la clase de prácticas.

Introducción

Esta práctica está diseñada para familiarizarse con los principales algoritmos de búsqueda utilizados en inteligencia artificial, tanto en entornos deterministas como en escenarios con adversarios. A lo largo de la práctica, se abordarán técnicas de búsqueda no informada, informada y estrategias competitivas aplicadas a juegos, permitiendo desarrollar agentes capaces de tomar decisiones eficientes. La práctica se divide en dos partes: la primera centrada en la resolución de problemas mediante búsqueda clásica, y la segunda en el diseño de heurísticas y estrategias para búsqueda con adversarios en juegos de dos jugadores. El lenguaje de programación de ambas partes es Python.

Propuesta de planificación

El enunciado tiene dos partes diferenciadas en base a las técnicas utilizadas. En concreto, en la segunda parte se realiza un torneo con entregas en días concretos, por lo que, se recomienda haber terminado la primera parte antes de que comiencen los torneos. Se propone la siguiente planificación:

- Semanas 1 y 2 (15 y 21 de septiembre): búsqueda informada y no informada.
- Resto de semanas (28 de septiembre, 5 y 12 de octubre): búsqueda con adversarios.

Método de entrega

Las prácticas se harán por parejas. Cada envío debe constar de un único archivo zip llamado **p1_gggg_mm_apellido1_apellido2.zip**, donde gggg es el número de vuestro grupo de laboratorio y mm el identificador asignado a cada persona/pareja en este grupo. Este archivo zip debe enviarse a través de Moodle.

Es importante, siempre, poner los nombres de los autores y el número del grupo de prácticas en todos los archivos que envíes: memoria y código.

Material a entregar

El fichero zip a entregar debe incluir:

- El *código* de la práctica, incluyendo los ficheros modificados con vuestro código, tal y como se explica en el enunciado. En esta carpeta deben aparecer completadas las secciones de código que en la versión entregada al alumno se indican como “***YOUR CODE HERE /TU CÓDIGO AQUÍ***”. Si habéis desarrollado ficheros adicionales (por ejemplo, para realizar pruebas o responder a alguna pregunta), debéis documentarlos e indicarlos en la memoria y en un fichero README.txt.
- Una memoria de la práctica (en formato PDF)
- Se recomienda que los resultados o comparaciones incluidas en la memoria se muestren **en forma de tablas o gráficas**. Es aconsejable que los razonamientos sobre complejidad no se hagan en base a un solo tablero o juego, sino, en la medida de lo posible, sean *razonamientos generales*, bien aplicando aspectos teóricos o habiendo revisado varios ejemplos.

La memoria a entregar no debe superar el límite máximo de 8 páginas y 2500 palabras.

Calificación

La práctica se evaluará dentro de un rango de 0-10, donde se tendrá en cuenta tanto la memoria como el código entregado (ver el apartado correspondiente de evaluación).

Deshonestidad académica: Comprobaremos vuestro código con otras entregas en la clase para verificar la redundancia lógica. Si copiáis el código de otra entrega y lo enviáis con cambios menores, lo sabremos. Estos detectores de trampas son bastante difíciles de engañar, así que no lo intentéis. Confiamos en que todos vosotros enviareis únicamente vuestro propio trabajo; *por favor* no nos defraudéis. Si lo hacéis, utilizaremos los recursos disponibles más severos.

Obtener ayuda: ¡No estás solo! Si os encontráis atascados en algo, comunicádnoslo a los profesores para poder obtener ayuda.

CONTEXTO

El juego del Reversi

Reversi (también conocido como **Othello** o **Yang**) es un juego de estrategia de dos jugadores. Se juega en un tablero de 8×8 con la siguiente configuración inicial:

	A	B	C	D	E	F	G	H
1								
2								
3								
4				○	●			
5				●	○			
6								
7								
8								

Los dos jugadores, representados por las fichas negras y blancas, alternan turnos, empezando las negras. En cada turno, el jugador coloca una pieza de su color sobre el tablero de forma que capture alguna de las fichas del contrario. Se realiza una captura cuando la nueva ficha, junto con alguna o algunas fichas del mismo color en el tablero, encierra hileras de fichas del contrario a lo largo de las filas, las columnas o las diagonales del tablero. Las fichas capturadas son sustituidas por fichas del color del jugador que realiza la captura. En caso de que el jugador que tiene el turno no pueda capturar ninguna ficha del contrario, el turno pasa al otro jugador.

El juego termina cuando ninguno de los jugadores puede capturar fichas del contrario o el tablero está completamente ocupado. El ganador de la partida es el jugador que tiene más fichas en el tablero.

En el siguiente ejemplo el jugador coloca la ficha ● en E6 (izquierda) y captura la ficha ○ que está en E5 (derecha).

	A	B	C	D	E	F	G	H
1								
2								
3								
4				○	●			
5				●	○			
6					●			
7								
8								

	A	B	C	D	E	F	G	H
1								
2								
3								
4				○	●			
5				●	●			
6					●			
7								
8								

Podéis jugar online en <https://webgamesonline.com/reversi/> para familiarizados con sus reglas y dinámicas.

Variante con obstáculos. Cuando el tablero incluye celdas bloqueadas (indicadas en la figura siguiente con un fondo gris), ningún jugador podrá utilizar dicha posición del tablero para colocar su ficha. Además, estas fichas bloquean las líneas de captura (evitando que se puedan capturar fichas *a través* de los obstáculos), a no ser que se aplique una regla de ignorar obstrucciones. Se puede comparar el efecto de esta regla en los siguientes tableros (salvo que se indique lo contrario, si un tablero incluye obstáculos, NO se aplicará esta norma de ignorar obstrucciones, por lo que estaremos en la situación de la izquierda).

	A	B	C	D	E	F	G	H
1								
2								
3								
4				○	●			
5				●	○			
6								
7								
8								

	A	B	C	D	E	F	G	H
1	●		○	○				
2	●	●	○	○	X	X		
3	●	○	○	●	○	X		
4	●	X	●	●	○	X		
5				●	○	X		
6						X		
7								
8								

Las celdas marcadas con X es donde el jugador con las fichas negras podría colocar su siguiente ficha teniendo en cuenta que los obstáculos bloquean las líneas de captura.

	A	B	C	D	E	F	G	H
1	●		○	○	X			
2	●	●	○	○	X	X		
3	●	○	○	●	○	X		
4	●	X	●	●	○	X		
5				●	○	X		
6						X		
7								
8								

En este caso, al aplicar la regla de ignorar obstrucciones, surge un movimiento más (marcado con X) ya que se permiten capturas a través de las celdas bloqueadas (en este caso, el obstáculo de B1).

PARTE A. Búsqueda no informada e informada

A.1 Búsqueda clásica

Para esta primera parte se aplicarán los problemas de búsqueda estudiados durante el curso donde un agente debe encontrar una secuencia de acciones que le permita alcanzar un objetivo. En estos problemas se asume que el **entorno es estático** y que la evolución del sistema depende **únicamente** de las decisiones tomadas por el agente. Sin embargo, muchos problemas reales implican la presencia de otros agentes cuyas acciones afectan al resultado final y deben ser tenidas en cuenta durante el proceso de planificación.

Antes de abordar la búsqueda con adversarios, resulta útil analizar cómo se representan los juegos como problemas de búsqueda. En un juego, cada estado describe una configuración posible del tablero y cada acción corresponde a un movimiento legal que puede realizar uno de los jugadores. A partir de un estado inicial es posible construir un árbol de juego en el que los nodos representan estados y las aristas representan movimientos.

En esta sección estudiaréis la estructura de estos árboles utilizando el juego del Reversi explicado anteriormente. El objetivo es comprender cómo crece el espacio de búsqueda y cómo las técnicas de búsqueda clásica pueden utilizarse para explorar diferentes secuencias de movimientos. Como simplificación, y dado que el problema de búsqueda debe tener un objetivo a alcanzar que no dependa del jugador actual, se considerará que cualquier estado en que se haya capturado una esquina, es un estado objetivo (ver clase *CornerReversiSearchProblem*).

A.1.1: Generación del árbol de juego

El código proporcionado incluye toda la funcionalidad necesaria para jugar una partida de Reversi y generar automáticamente los movimientos legales disponibles desde cada posición, sin embargo, el estado de juego como tal se entrega parcialmente implementado (*CornerReversiState*, en [search.py](#)).

Completad la clase *CornerReversiState* donde se indica “YOUR CODE HERE”. Para ello, conviene revisar los métodos estáticos en [reversi.py](#) por si fueran útiles.

Una vez hecho eso, implementad la función *build_game_tree* (en [search.py](#)) que, dado un *CornerReversiState* instanciado a partir de un tablero de Reversi, genere un árbol de juego hasta una profundidad máxima *d* dada. La función deberá contabilizar:

- Número total de nodos generados.
- Número de nodos hoja.
- Factor de ramificación medio observado.
- Profundidad máxima alcanzada.

Probad vuestro código (basta con lanzarlo como “python search.py”) utilizando las profundidades 1, 2, 3 y 4 sobre el tablero inicial de Reversi.

Pregunta 1.1: ¿Cómo evoluciona el número de nodos al aumentar la profundidad?

Pregunta 1.2: ¿Es viable explorar exhaustivamente el árbol completo de Reversi hasta el final de la partida? Justifica tu respuesta.

A.1.2: Exploración mediante búsqueda clásica

En este apartado, vamos a utilizar algoritmos de búsqueda clásicos para recorrer el árbol de búsqueda. En particular, partiendo de la versión incluida en el código (fichero [search.py](#)) de Búsqueda en profundidad (BP), completad esa implementación y cread otra función que realice Búsqueda en anchura (BA). Para ello, tendréis que crear un **estado de búsqueda** que incluya, por lo menos:

- información sobre la etapa actual en la búsqueda,
- información suficiente para que sea posible tener acceso a la trayectoria desde el principio hasta el final.

Estas funciones de búsqueda devolverán la lista de acciones que llevan al agente desde el inicio hasta el objetivo. Estas acciones deben ser movimientos legales. En el caso de que la búsqueda falle (el algoritmo no encuentra solución), se debe devolver None.

Además, para facilitar el análisis y comparación entre los métodos de búsqueda, junto con la lista de acciones, la función de búsqueda debe devolver el número de nodos visitados. También es aconsejable revisar el fichero [utils.py](#) ya que incluye estructuras que pueden ser útiles para este u otros apartados.

Aplicad ambos algoritmos sobre distintos tableros iniciales (no es necesario generar como tal el árbol de búsqueda) e instrumentad el código para medir:

- Número de nodos visitados.
- Orden de exploración de los nodos.

Completa el fichero [search.py](#) para que se obtenga esta información simplemente haciendo “python search.py”.

Pregunta 1.3: ¿Qué diferencias observáis entre BP y BA al explorar árboles de juego?

Pregunta 1.4: ¿Cuál de los dos algoritmos requiere más memoria? Explica por qué.

Pregunta 1.5: Suponed que cada hoja contiene una puntuación fija (por ejemplo, en función del número de fichas negras). ¿Permiten estos algoritmos determinar cuál es la mejor jugada para un jugador racional? Justificad vuestra respuesta.

A.2: Aplicando búsqueda informada

Los métodos de búsqueda estudiados hasta ahora permiten explorar un espacio de estados y localizar soluciones solo en función de cuándo se descubrieron. Supongamos ahora que queremos elegir una jugada sin explorar hasta el final de la partida. ¿Cómo podemos estimar qué estados parecen más prometedores?

Para hacer estas estimaciones, tenemos que definir funciones de evaluación (**heurísticas**) y utilizar métodos de búsqueda que usen esta información. Si bien perderemos generalidad (las heurísticas utilizarán información de cada problema a resolver), deberían permitir encontrar la solución más rápidamente.

Implementad una función en **search.py** que realice Búsqueda A*, tomando una función heurística como argumento (es decir, completad la función *aStarSearch* que se entrega en el código). La heurística toma dos argumentos: un estado en el problema de búsqueda (el argumento principal) y el problema en sí (como información de referencia).

Pregunta 1.6: Define 3 funciones heurísticas con varios niveles de complejidad (puedes empezar a partir de la heurística entregada *simpleHeuristic*), implementando las funciones ya definidas *heuristic1*, *heuristic2*, *heuristic3* (de menor a mayor complejidad): ¿qué características parecen ayudar a resolver el problema de capturar una esquina en un menor número de pasos?

Pregunta 1.7: Analiza qué ocurre con los métodos de búsqueda y las distintas heurísticas propuestas si se intenta resolver el problema de capturar todas las esquinas, es decir, en el *main*, instancia el problema *AllCornersReversiSearchProblem* en lugar de *CornerReversiSearchProblem*.

Pregunta 1.8: En base a tus resultados, ¿puede la función heurística equivocarse? ¿qué aspectos habría que tener en cuenta para hacer que A* sea óptimo?

PARTE B. Búsqueda con adversarios

En un juego competitivo existe una diferencia fundamental con respecto a lo planteado hasta ahora: no controlamos todas las decisiones que determinan la evolución del sistema. Después de realizar una jugada, el adversario elegirá también la acción que considere más beneficiosa para sus intereses.

Esto implica que ya no basta con buscar una secuencia de movimientos favorable. Una estrategia óptima debe considerar las posibles respuestas del oponente y asumir que este actuará de manera racional para maximizar su propia ventaja. Como consecuencia, las hojas de un árbol de juego no representan simplemente estados alcanzables, sino resultados que dependen de las decisiones de ambos jugadores.

Para modelar esta situación se utilizan árboles de búsqueda adversarial. En ellos se distinguen dos tipos de nodos:

- **Nodos MAX**, en los que juega nuestro agente y trata de maximizar la utilidad obtenida.
- **Nodos MIN**, en los que juega el adversario y trata de minimizar dicha utilidad.

El objetivo consiste en seleccionar la acción que conduce al mejor resultado posible bajo la hipótesis de que ambos jugadores actuarán racionalmente.

B.1 Algoritmo Minimax

Se proporciona una implementación del algoritmo *minimax* (ver clase MinimaxStrategy en el módulo [strategy.py](#)). Esta implementación está basada en el siguiente pseudocódigo, donde, entre otros aspectos, se considera una profundidad máxima como parámetro y se comprueba la profundidad, no solo si un estado es terminal, para devolver el valor de utilidad:

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow \infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
  return v
```

Para realizar esta parte de la práctica, define una clase de nombre `MinimaxAlphaBetaStrategy` que herede de `Strategy` (puedes utilizar la clase `MinimaxStrategy` como punto de partida) que implemente el algoritmo minimax con poda alfa-beta. La implementación debe ser acorde con este pseudocódigo:

```
function ALPHA-BETA-DECISION(state) returns an action
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
          $\alpha$ , the value of the best alternative for MAX along the path to state
          $\beta$ , the value of the best alternative for MIN along the path to state
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for a, s in SUCCESSORS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$ 
    if  $v \geq \beta$  then return v
     $\alpha \leftarrow \text{MAX}(\alpha, v)$ 
  return v

function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  same as MAX-VALUE but with roles of  $\alpha$ ,  $\beta$  reversed
```

B.2 Funciones de evaluación heurística para búsqueda con adversarios

En esta sección, trabajaremos en el diseño y definición de funciones heurísticas que tengan en cuenta el estado del juego y tomen decisiones en función de qué jugador tiene el control en cada momento. Para averiguar su utilidad, las enfrentaremos en varios torneos, para lo cual, deben cumplir una serie de restricciones.

Para probar vuestras funciones de evaluación heurística, debéis crear un fichero con:

1. Tantas clases que hereden de `StudentHeuristic` (en **`tournament.py`**) como funciones de evaluación queráis probar (ver ayuda del código en el Apéndice 1).
2. Dentro de cada clase, tenéis que implementar dos funciones: una de nombre `evaluation_function` (que recibe un estado de tipo `TwoPlayerGameState` representando un potencial movimiento en la partida, y debe devolver una estimación de su interés para el jugador, cuando mayor el valor, mayor el interés) y otra `get_name` donde indicaréis el nombre con el que esta heurística aparecerá en el torneo.

A continuación, tenéis un ejemplo con una heurística trivial (no hace nada con el estado), donde podéis ver cómo se pueden incluir funciones auxiliares:

```

from game import (
    TwoPlayerGameState,
)
from tournament import (
    StudentHeuristic,
)

class MySolution1(StudentHeuristic):
    def get_name(self) -> str:
        return "mysolution1"
    def evaluation_function(self, state: TwoPlayerGameState) -> float:
        # let's use an auxiliary function
        aux = self.dummy(123)
        return aux

    def dummy(self, n: int) -> int:
        return n + 4

class MySolution2(StudentHeuristic):
    ...

```

Además de probar las funciones con los programas que se entregan en el proyecto, a través de una interfaz web, podréis subir **el único fichero python** que se os pide para que podáis definir hasta un máximo de 3 estrategias que se usarán en los torneos.

Importante: el número de heurísticas para participar en los torneos está limitado a **3**, por lo que, aunque incluyáis más clases, no se tendrán en cuenta en el torneo (no debéis asumir qué 3 heurísticas se usarán). El nombre de este fichero debe ser único y que identifique a cada pareja: *p1_gggg_mm_apellido1_apellido2.py*.

Observaciones:

- No se admitirán funciones que hagan llamadas a funciones del sistema, usen funciones de control del juego o manipulen sus estructuras de datos.
- Es importante que la función de evaluación sea eficiente, ya que se controlará el tiempo invertido por cada jugador, de manera que se dará la partida por perdida a aquel jugador que sea muy lento. Este baremo de cálculo no podrá ser un tiempo absoluto (que depende, además de la máquina, de diversos factores, entre ellos del SO utilizado y de la profundidad de la búsqueda). No obstante, para dar una indicación que sea reproducible en cualquier máquina, habrá que tener en cuenta que **aquellos jugadores que tarden más que 10 veces lo que tarda la heurística trivial** (clase `Heuristic1` en `demo_tournament.py`) en un tablero 8x8 muy probablemente serán eliminados por *timeout*. No es posible asegurar que siempre serán eliminados, porque la duración de una evaluación depende también del nº de movimientos posibles, lo que introduce cierta variabilidad.

Evaluación

Para la calificación de esta práctica, se tendrán en cuenta los siguientes 4 aspectos.

Calificación de funciones de búsqueda (2 puntos)

Como se indica en la parte A del enunciado, hay que implementar una función para construir un árbol de juego, tres algoritmos de búsqueda (BP, BA, A*) y tres funciones heurísticas. El reparto de puntos es: 0.25 (BP), 0.5 (BA), 1.25 (A* y heurísticas).

Aunque sólo se entrega código para probarlo con problemas relacionados con Reversi, los algoritmos de búsqueda deberían ser generales y no depender de cómo está diseñado el juego o el estado de Reversi.

Calificación de funciones de evaluación heurística (3 puntos)

Consiste en la implementación y presentación de funciones de evaluación heurística a un torneo entre todas las parejas de estudiantes de la asignatura. Los estudiantes dispondrán de 10 tokens al inicio de la práctica, donde cada ejecución de un torneo interno consumirá un token. Estos torneos internos permitirán averiguar cómo se comportan las heurísticas implementadas frente a funciones heurísticas de varios niveles provistas por los profesores. En los torneos se probarán **tableros 8x8** comenzando con dos estados iniciales: por defecto y con obstáculos en B1, H2, A7 y G8. Estos torneos se lanzarán a **profundidades de minimax no mayores de 4**, por lo que es recomendable realizar pruebas con esta configuración.

Torneo general (1.5 puntos)

Como resultado de los enfrentamientos entre funciones de evaluación se publicará una clasificación, accesible mediante un enlace que se mostrará en la parte del torneo del curso Moodle. En las fechas señaladas más abajo, se realizará un torneo entre las estrategias que estén subidas y se actualizará el ranking en la página de Moodle habilitada para ello. En cierto momento se introducirán las heurísticas de referencia proporcionadas por los profesores, las cuales corresponden a las de los torneos internos y servirán de indicadores de nivel de juego.

Se recomienda a los estudiantes ejecutar torneos internos contra las heurísticas de referencia de los profesores (usando los tokens que tengan disponibles) antes de que se realicen los torneos generales, para obtener una estimación del rendimiento de sus propias heurísticas. Estos torneos se explican en la siguiente sección.

La nota de esta parte se calculará de la siguiente manera:

Nota obtenida	Percentil X en el ranking
150,00 %	$X \geq 90$
100,00 %	$90 > X \geq 50$
75,00 %	$50 > X \geq 10$
50,00 %	$10 > X$

Cada torneo tendrá un peso que se irá incrementando: **15, 35 y 50%**.

Habrán torneos abiertos los siguientes días, antes de cuya fecha se tendrá que subir 3 funciones (al menos una DEBE ser distinta a las enviadas anteriormente o habrá una penalización):

- Torneo 1: Miércoles **7 de octubre** (subir antes de las 20:00)
- Torneo 2: Miércoles **14 de octubre** (subir antes de las 20:00)

Estas dos entregas son obligatorias y **será penalizada la no entrega** (con un 0 en el ranking correspondiente por cada entrega que falte). La entrega final de funciones de evaluación será el día:

- Torneo 3: Miércoles **21 de octubre** (subir antes de las 20:00).

La participación a los torneos se hace separada de la entrega completa de la P1 (la cual se hace por Moodle) en las fechas indicadas al comienzo de este enunciado. Para los torneos se usará la interfaz web indicada en Moodle.

Para entrar a este sistema, deberás usar tu dirección de correo electrónico y una contraseña que podrás encontrar en la parte de retroalimentación de la tarea de Moodle con nombre *“Acceso para usar sistema de torneo en la P1”*.

Importante: los envíos están controlados por parejas de prácticas, es decir, que los dos miembros de la pareja participan con el mismo envío de heurística, por lo que es crítico que el sistema tenga esta información actualizada. Si el sistema muestra a los dos miembros de la pareja información diferente sobre su grupo de prácticas o número de pareja, informad al coordinador de prácticas.

En cualquier caso, tanto en el código como en la memoria entregados se volverán a incluir las estrategias seleccionadas y se analizarán en detalle las distintas soluciones presentadas, incluyendo referencias a artículos (siguiendo el formato APA¹) que expliquen o justifiquen las heurísticas planteadas.

¹ APA Quick Citation Guide (visitada por última vez en febrero 2022):

<https://guides.libraries.psu.edu/apaquickguide/intext>.

Observación: se pueden obtener referencias en formato APA muy fácilmente buscando el artículo en cuestión en Google Scholar, y luego pulsando en el botón de “Cite”. Por ejemplo, al hacer eso con la siguiente URL: <https://scholar.google.com/scholar?q=the%20texbook>, eligiendo el formato APA nos devuelve:

Knuth, D. E., & Bibby, D. (1984). *The texbook* (Vol. 15). Reading: Addison-Wesley.

Torneo interno.

Los estudiantes pueden ejecutar torneos internos utilizando los tokens asignados al inicio de la práctica. Cada ejecución de un torneo interno consume un token. Hay un total de 10 tokens disponibles por pareja al inicio de la práctica. En los torneos internos, las heurísticas entregadas por los estudiantes se enfrentarán a las heurísticas de referencia proporcionadas por los profesores, que también participan en los torneos generales, y a una heurística random (ver siguiente apartado). Un uso estratégico de los tokens permitirá obtener información valiosa para ajustar y mejorar las heurísticas antes de competir en los torneos generales.

Enfrentamientos con random (1.5 puntos)

Las tres heurísticas entregadas en el **último torneo** se enfrentarán 1000 veces contra una heurística trivial que asigna valores aleatorios. Estos enfrentamientos se realizarán utilizando minimax con una profundidad de 1. La nota asociada a estos enfrentamientos se calculará de la siguiente manera:

$$nota = 1.5 \cdot \max\left(0, \left(\frac{\# \text{ victorias}}{1000} - 0.5\right) \cdot 2\right)$$

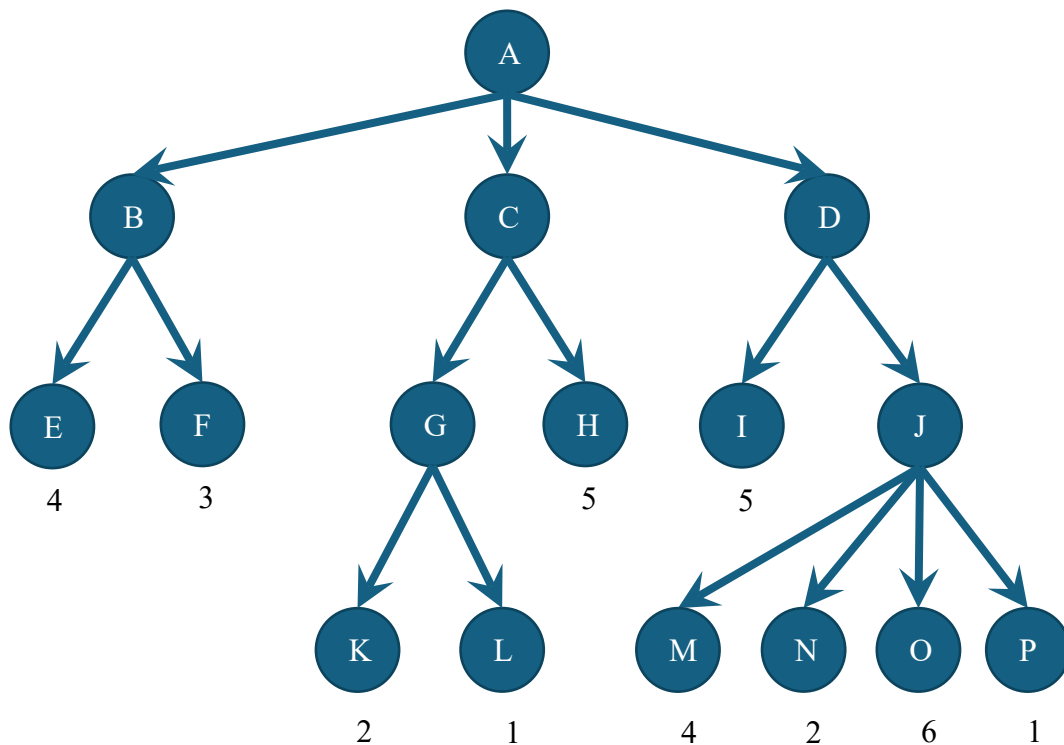
Se recomienda comprobar el comportamiento de las heurísticas desarrolladas con una heurística trivial, ya que ganar un 50% o menos de las veces contra ella indica que vuestra heurística toma peores decisiones que si simplemente evaluara los tableros al azar.

Calificación de algoritmo minimax con poda (2 puntos)

El código entregado se valorará tanto por funcionalidad como por estilo. Es importante documentar en la memoria de forma clara el algoritmo empleado en la función realizada, así como su funcionamiento y sus características más relevantes. Para justificar que la poda está funcionando, se debe entregar un informe de tiempos, para lo cual, se aconseja el uso de la librería `timeit`². Para ser más específicos y evitar variaciones debidas a jugadores no deterministas, se realizará el análisis con un jugador que utilice la estrategia determinista mencionada en la Sección B.2, es decir, *Heuristic1* en `demo_tournament.py` a profundidades 3 y 4. Además, deberías realizar el mismo análisis con una heurística tuya y comparar los resultados, incluyendo la discusión en la memoria. **Ten en cuenta que los tiempos pueden depender del ordenador en el que trabajas, por lo que, además de los tiempos absolutos, deberíais proporcionar alguna medida de la mejora que sea independiente de dicho ordenador.**

Con el fin de determinar si las implementaciones de minimax y minimax con poda alfa-beta son correctas, se propone considerar el juego definido por el siguiente árbol:

²Documentación de `timeit` (visitada por última vez en Septiembre 2024):
<https://docs.python.org/3/library/timeit.html>



La especificación del juego se encuentra en el fichero simple_game_tree.py.
Para este juego:

- (i) Aplica manualmente el algoritmo minimax. Sobre una copia del árbol, identifica el número de iteración, el nodo que se está visitando y la estimación del valor minimax para dicho nodo en cada paso del algoritmo.
- (ii) Comprueba que la ejecución manual coincide con la realizada por el jugador minimax en demo_simple_game_tree.py que inicia la partida. Para ello, indica en la memoria la información correspondiente al primer movimiento del juego con verbose=3.
- (iii) Aplica manualmente el algoritmo minimax con poda alfa-beta. Identifica el número de iteración, el nodo que se está visitando y la estimación del intervalo $[a, b]$ para dicho nodo en cada paso del algoritmo. Finalmente, indica el valor minimax en el nodo raíz y la secuencia de movimientos hasta el final de la partida.
- (iv) Comprueba que la ejecución manual coincide con la realizada por el jugador minimax que utiliza poda alfa-beta en demo_simple_game_tree.py. que inicia la partida. Para ello, **indica en la memoria la secuencia de valores del intervalo $[a, b]$ asociados a los nodos visitados en cada iteración del algoritmo.**

Memoria (3 puntos)

Consiste en un documento con una descripción de las funciones de búsqueda y evaluación implementadas, así como la documentación asociada a la implementación del algoritmo alfa-beta, que se subirá a Moodle junto con el resto del código. **El informe debe ser claro, completo y conciso. Por favor, incluye sólo información relevante.**

1. (1 punto) Respuesta a las preguntas de la parte A.
2. (1 punto) Documentación de minimax y la poda alfa-beta
 - a. Detalles de implementación: ¿qué tests se han diseñado y aplicado para determinar si la implementación es correcta?, diseño e implementación (estructuras de datos seleccionadas, descomposición funcional, etc.), otra información relevante.
 - b. Eficiencia de la poda alfa-beta: descripción completa del protocolo de evaluación, tablas donde se incluyan tiempos con y sin poda, medidas de mejora independientes del ordenador, análisis correcto, completo y claro de los resultados, otra información relevante.
3. (1 punto) Documentación del diseño de la heurística.
 - a. Revisión de trabajos previos sobre estrategias de Reversi, incluyendo referencias en el formato APA.
 - b. Descripción del proceso de diseño: ¿cómo se planeó y ejecutó el proceso de diseño? ¿seguiste algún procedimiento sistemático para evaluar las heurísticas diseñadas? ¿cuál fue el rendimiento de tus heurísticas contra las heurísticas de referencia? ¿y contra random? ¿te apoyaste en esos resultados para plantear modificaciones? ¿utilizaste ideas desarrolladas por otros para mejorar las estrategias diseñadas? Si están disponibles públicamente, incluye referencias en formato APA; en otro caso, incluye el nombre de la persona que te dio la información y dale el crédito oportuno como “comunicación privada”.
 - c. Descripción de la heurística enviada finalmente.
 - d. Otra información relevante.

Apéndice 1: Código de la práctica

El código proporcionado requiere la versión 3.10 de Python, aunque recomendamos usar la 3.12. La infraestructura para un juego entre dos jugadores está en el módulo [game.py](#). Las clases y funciones específicas para Reversi están implementadas en el fichero [reversi.py](#). **Estos ficheros no deben ser modificados.**

Los prototipos de las funciones de búsqueda, el estado de Reversi para búsqueda, así como los dos problemas de búsqueda definidos en la Sección A están implementados o indicados en [search.py](#). En este fichero tendrás que incluir tus implementaciones y podrás modificar el main para hacer pruebas y responder a las preguntas correspondientes.

Las estrategias para el juego (jugador aleatorio, jugador manual, jugador minimax) están implementadas en [strategy.py](#). **Este es el fichero que se debe modificar para implementar un jugador que realice una búsqueda minimax con poda alfa-beta (ver Sección B.1).**

En el módulo [heuristic.py](#) se ha implementado un ejemplo de función de evaluación de estados del juego, la cual es utilizada en estrategias de búsqueda. Esta función es general al no usar conocimiento del juego en cuestión, por lo que tiene bastante margen de mejora, sin embargo, por ello mismo se puede usar la misma heurística para varios juegos.

Podéis encontrar ejemplos de cómo jugar en el fichero [demo_reversi.py](#).

En los distintos ejemplos que podéis ver en este fichero, se observa lo siguiente:

- La lógica del juego (en este caso, Reversi) se abstrae dentro de una clase de tipo `TwoPlayerGameState`, que almacena la información necesaria para juegos de dos jugadores (además de la lógica del juego, el tablero, el jugador inicial, si se ha terminado el juego, las puntuaciones, etc.)
- La lógica de una partida de dos jugadores se maneja a través de la clase `TwoPlayerMatch`, que recibe un estado de tipo `TwoPlayerGameState` y es el encargado de dar turno a cada jugador, imprimir más o menos información por pantalla (según el valor de la variable `verbose`), comprobar que ningún jugador invierta mucho tiempo en cada turno (ver variable `max_seconds_per_move`), así como de controlar si la interfaz será textual o gráfica (variable `gui`).

Para terminar de entender el código entregado, se explican a continuación las clases y funcionalidades más importantes:

- El parámetro `gui` de la clase `TwoPlayerMatch` sirve para decidir si el juego se mostrará como texto en la terminal o si se creará una ventana gráfica. En cualquiera de los dos casos se pueden usar cualquiera de los tipos de jugadores (manual o automático). Si el jugador es manual, el comportamiento es diferente: en el caso de que se haya elegido jugar con la interfaz gráfica (el valor de la

variable `gui` es `True`), entonces el jugador tendrá que hacer clic en la ventana; en modo texto (`gui=False`) realizará su jugada por la terminal.

- La variable `verbose > 0` permite obtener información sobre la búsqueda minimax. En concreto,
 - `verbose = 1` # Se imprime el valor minimax de la jugada seleccionada
 - `verbose = 2` # Se ejecuta el juego paso a paso y se imprimen los valores minimax de todos los estados explorados en la búsqueda.
 - `verbose = 3` # Se ejecuta el juego de manera continua y se imprimen los valores minimax de todos los estados explorados en la búsqueda.

Adicionalmente, en los modos con $0 < \text{verbose} < 3$ se ofrece la posibilidad de almacenar el estado intermedio correspondiente; es decir la información sobre el jugador que va a realizar el movimiento y el estado del tablero.

- En [demo_reversi.py](#) podéis encontrar distintos jugadores inicializados: manuales, aleatorios, que utilizan estrategias con búsqueda (aplicando el algoritmo minimax para la búsqueda y una función para la evolución de estados del juego). Los jugadores se implementan como estrategias de juego (clase abstracta `Strategy` definida en el módulo [strategy.py](#)). También se puede ver cómo crear un jugador que utilice una heurística creada por vosotros.
- Podéis también explorar otros juegos más simples (puede ser útil para depurar el código): [demo_tictactoe.py](#) y [demo_simple_game_tree.py](#).
- En el módulo [heuristic.py](#) se incluye una función heurística (`simple_evaluation_function`) que se puede pasar a jugadores de tipo minimax. En el siguiente apartado veréis que **uno de vuestros objetivos en esta práctica es diseñar una función heurística que sea competitiva**.
- El tipo de estrategia `MinimaxStrategy`, además de la función heurística, recibe como argumento la máxima profundidad a la que se hará la búsqueda (`max_depth_minimax`).
- Para permitir experimentar más rápido y probar distintas situaciones, la infraestructura entregada permite jugar a partir de un estado intermedio. Para ello es necesario inicializar las variables `initial_board` e `initial_player` con los valores correspondientes. Para jugar a partir de la configuración estándar se puede utilizar `board = None` en la instanciación de un objeto de tipo `TwoPlayerGameState`.
- Las funciones `from_dictionary_to_array_board`, `from_array_to_dictionary_board` que se encuentran en el fichero `reversi.py` permiten transformar el tablero en forma de diccionario (el usado en la implementación del juego) a uno en forma de array (más fácil de visualizar), y viceversa.
- Los ficheros [tictactoe.py](#) y [demo_tictactoe.py](#) permiten jugar al Tres en Raya, un juego más sencillo que Reversi y con menor factor de ramificación, que puede ser útil para vuestras pruebas.
- Para la implementación del algoritmo minimax con poda alfa-beta, puede ser útil implementar un juego más sencillo o analizar alguno de los que se dan en la práctica. Por ejemplo, uno definido por un árbol de juego (ver [simple_game_tree.py](#) y [demo_simple_game_tree.py](#)).

Además de probar a jugar las partidas que están configuradas en el módulo `demo_reversi.py`, también podéis llevar a cabo un torneo con varias estrategias y funciones de evaluación llamando a la función `run` del módulo `tournament.py` indicando:

1. Las estrategias.
2. El número de partidas con fichas negras (se jugará el mismo número con blancas).
3. Los nombres o alias de las estrategias.

Una posible salida del código que tenéis en `demo_tournament.py` es la siguiente:

	total:	opt1_dummy	opt2_random	opt3_heuristic
opt1_dummy	2:	---	2	0
opt2_random	9:	8	---	1
opt3_heuristic	19:	10	9	---

En esta configuración se han jugado 30 partidas en total (20 para cada estrategia): $30 = 5 \times 2 \times 3 = 5$ repeticiones (parámetro `n_pairs`) $\times$ 2 jugadores (cada estrategia juega como blancas y negras) $\times$ número de estrategias (3 en este caso). Se puede ver que la estrategia `random` gana 9 de los 30 juegos, 8 contra `dummy` y 1 contra `heuristic`.

En el mismo fichero podéis ver (entre comentarios) cómo cargar estrategias desde un fichero que exista en una carpeta, tal y como se cargarán en el torneo. Puede ser útil probarlo de esta manera para confirmar que funciona sin problemas.

Observación: la función `run` del módulo `tournament.py` usa `MinimaxStrategy`, sin embargo, cuando tengáis implementado `MinimaxAlphaBetaStrategy`, es aconsejable que lo cambiéis para ahorrar tiempo en vuestras ejecuciones.